Laporan Tugas Kecil 2

Strategi Algoritma (IF2211)

Disusun Oleh:

Mirza Tsabita Wafa'Ana (13524114)

Program Studi Teknik Informatika

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

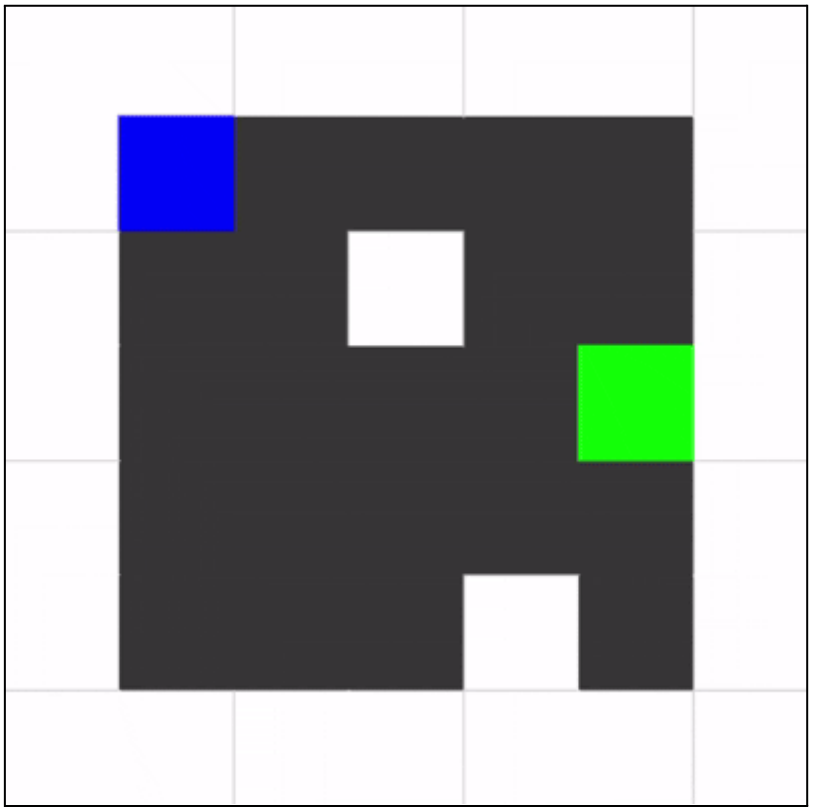
INSTITUT TEKNOLOGI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI 2
BAB 1 DESKRIPSI TUGAS 3
BAB 2 PENYELESAIAN ALGORITMA 3
2.1 Algoritma Pathfinding yang Digunakan 3
2.2 UCS 4
2.3. GBFS 4
2.4. A* 4
2.5. Bonus BFS 5
2.6. Definisi f(n), g(n), dan h(n) 5
2.7. Apakah Heuristic A* Admissible? 5
2.8. Perbandingan Teoritis Algoritma 5
2.9. Source Program 6
BAB 3 SCREENSHOT INPUT OUTPUT 16
3.1. TestCase 1 16
3.2. TestCase 2 18
3.3. TestCase 3 20
3.4. TestCase 4 21
3.5. TestCase 5 22
3.6. Analisis Hasil Percobaan 22
4.1. Link Repository Program 23
4.2. Pernyataan Tidak Melakukan Kecurangan 23

BAB 1 DESKRIPSI TUGAS



Visualisasi Permainan Ice Sliding Puzzle Solver

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih).

BAB 2 PENYELESAIAN ALGORITMA

2.1 Algoritma Pathfinding yang Digunakan

Program ini menggunakan empat algoritma pencarian jalur, yaitu **UCS**, **GBFS**, **A***, dan **BFS** sebagai tambahan. Algoritma tersebut digunakan untuk mencari jalur dari posisi awal Z menuju titik tujuan O pada permainan Ice Sliding Puzzle.

Pada permainan ini, aktor bergerak dengan cara meluncur terus ke satu arah sampai berhenti sebelum batu X. Aktor tidak dapat berhenti di tengah jalan seperti permainan maze biasa. Selain itu, aktor tidak boleh melewati lava L dan harus melewati angka 0, 1, 2, ... secara berurutan sebelum mencapai tujuan.

Setiap kondisi permainan direpresentasikan sebagai **state**. State terdiri dari posisi aktor saat ini dan angka berikutnya yang perlu dilewati. Dengan penyimpanan seperti ini, posisi yang sama tetap dapat dianggap berbeda apabila progress angka yang sudah dilewati berbeda.

Program memproses setiap state dengan mencoba empat arah gerakan, yaitu atas, bawah, kiri, dan kanan. Ketika satu arah dipilih, aktor akan meluncur sampai berhenti sebelum batu. Jika gerakan tersebut valid, posisi baru akan dimasukkan ke proses pencarian sesuai algoritma yang dipilih.

2.2 UCS

Uniform Cost Search atau UCS adalah algoritma pencarian yang memilih state dengan total biaya terkecil terlebih dahulu. Pada program ini, UCS menggunakan PriorityQueue, sehingga state dengan cost paling kecil akan diproses lebih awal.

Proses UCS dimulai dari posisi awal dengan cost 0. Setelah itu, program mengambil state dengan cost terkecil, lalu mencoba seluruh kemungkinan gerakan ke empat arah. Jika gerakan valid, program menghitung cost baru dari total cost sebelumnya ditambah cost tile yang dilewati selama aktor meluncur. Jika state baru belum pernah dicapai atau ditemukan dengan cost yang lebih kecil, state tersebut dimasukkan kembali ke PriorityQueue.

UCS cocok digunakan pada permainan ini karena setiap tile memiliki cost berbeda. Dengan begitu, UCS dapat mencari jalur dengan total cost yang rendah.

2.3. GBFS

Greedy Best First Search atau GBFS adalah algoritma pencarian yang memilih state berdasarkan nilai heuristic terkecil. Heuristic digunakan sebagai perkiraan seberapa dekat posisi aktor dengan titik tujuan.

Berbeda dari UCS, GBFS tidak terlalu memperhatikan total cost yang sudah dikeluarkan. GBFS lebih mengutamakan posisi yang terlihat paling dekat dengan tujuan. Karena itu, GBFS sering kali dapat menemukan solusi lebih cepat, tetapi hasilnya belum tentu memiliki total cost paling kecil.

Pada program ini, GBFS menggunakan PriorityQueue dengan nilai prioritas dari heuristic. Semakin kecil nilai heuristic suatu state, semakin cepat state tersebut diproses.

2.4. A*

A* adalah algoritma pencarian yang menggabungkan total cost dari awal dan perkiraan jarak menuju tujuan. Dengan cara ini, A* tidak hanya memperhatikan jalur yang murah sejauh ini, tetapi juga mempertimbangkan arah menuju tujuan.

Rumus yang digunakan pada A* adalah:

$$f(n) = g(n) + h(n)$$

g(n) adalah total cost dari posisi awal sampai state saat ini. h(n) adalah nilai heuristic dari state saat ini menuju tujuan. f(n) adalah nilai prioritas yang digunakan untuk menentukan state mana yang diproses lebih dulu.

Pada program ini, A* menggunakan PriorityQueue dan memproses state dengan nilai f(n) terkecil.

2.5. Bonus BFS

Breadth-First Search atau BFS digunakan sebagai algoritma tambahan. BFS bekerja dengan prinsip FIFO, yaitu state yang masuk lebih awal akan diproses lebih dulu.

BFS dimulai dari posisi awal, lalu mencoba seluruh kemungkinan gerakan ke empat arah. Setiap state valid yang belum pernah dikunjungi akan dimasukkan ke dalam queue. Proses ini dilakukan sampai aktor mencapai titik tujuan.

BFS mencari solusi berdasarkan jumlah gerakan paling sedikit, bukan berdasarkan total cost. Karena itu, BFS dapat digunakan sebagai pembanding terhadap algoritma lain, tetapi tidak selalu menghasilkan jalur dengan cost paling rendah.

2.6. Definisi f(n), g(n), dan h(n)

g(n) adalah total cost dari posisi awal sampai state n. Pada program ini, nilai g(n) dihitung dari jumlah cost seluruh tile yang dilewati selama proses sliding. Tile awal pada satu gerakan tidak dihitung, tetapi tile yang dilewati dan tile tempat aktor berhenti tetap dihitung.

h(n) adalah nilai heuristic, yaitu perkiraan jarak dari posisi aktor saat ini menuju titik tujuan. Program menggunakan beberapa heuristic, yaitu Manhattan Distance, Euclidean Distance, dan Chebyshev Distance.

f(n) adalah nilai prioritas pada algoritma A*. Nilainya diperoleh dari penjumlahan g(n) dan h(n).

Pada UCS, prioritas hanya menggunakan g(n). Pada GBFS, prioritas hanya menggunakan h(n). Pada A*, prioritas menggunakan f(n).

2.7. Apakah Heuristic A* Admissible?

Heuristic utama yang digunakan adalah Manhattan Distance. Heuristic ini menghitung jarak berdasarkan selisih baris dan kolom antara posisi aktor dan titik tujuan.

Manhattan Distance dapat dianggap admissible karena hanya memberikan estimasi jarak sederhana menuju tujuan dan tidak melebih-lebihkan biaya perjalanan sebenarnya. Heuristic ini tidak memperhitungkan batu, lava, checkpoint angka, maupun cost tile secara detail. Karena itu, nilainya cenderung menjadi perkiraan bawah dari usaha yang benar-benar diperlukan.

Namun, karena permainan ini menggunakan mekanisme sliding dan memiliki cost tile yang berbeda-beda, nilai heuristic tidak selalu menggambarkan kondisi sebenarnya dengan sempurna. Walaupun begitu, Manhattan Distance tetap cukup aman digunakan sebagai estimasi karena tidak secara sengaja memperbesar nilai biaya menuju tujuan.

2.8. Perbandingan Teoritis Algoritma

Secara teori, UCS unggul dalam pencarian jalur dengan total cost rendah karena selalu memilih state dengan cost terkecil. Kekurangannya, UCS dapat memproses cukup banyak state karena tidak memiliki arahan heuristic menuju tujuan.

GBFS biasanya lebih cepat dalam menemukan jalur karena memilih state yang terlihat paling dekat dengan tujuan. Namun, GBFS tidak menjamin total cost yang rendah karena hanya mempertimbangkan heuristic.

A* lebih seimbang karena menggabungkan total cost dan heuristic. Jika heuristic yang digunakan cukup baik, A* dapat menemukan solusi yang efisien dengan jumlah iterasi yang lebih terkendali.

BFS mencari jalur berdasarkan jumlah gerakan paling sedikit. Akan tetapi, BFS tidak mempertimbangkan cost setiap tile, sehingga jalur dengan langkah sedikit belum tentu memiliki biaya paling rendah.

2.9. Source Program

2.9.1 Board.java

```
import java.io.File;
import java.util.Scanner;

public class Board {
    int n, m;
    char[][] grid;
    int[][] cost;

    int startRow, startCol;
    int goalRow, goalCol;
    int maxNumber = -1;

    public Board(String filename) throws Exception {
        readFile(filename);
    }

    void readFile(String filename) throws Exception {
        Scanner sc = new Scanner(new File(filename));

        n = sc.nextInt();
        m = sc.nextInt();

        grid = new char[n][m];
        cost = new int[n][m];

        boolean adaStart = false;
        boolean adaGoal = false;

        for (int i = 0; i < n; i++) {
            String line = sc.next();

            for (int j = 0; j < m; j++) {
                char c = line.charAt(j);
                grid[i][j] = c;

                if (c == 'Z') {
                    startRow = i;
                    startCol = j;
                    adaStart = true;
                } else if (c == 'O') {
                    goalRow = i;
                    goalCol = j;
                    adaGoal = true;
                } else if (Character.isDigit(c)) {
```

```
                maxNumber = Math.max(maxNumber, c - '0');
            }
        }
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cost[i][j] = sc.nextInt();
        }
    }

    sc.close();

    if (!adaStart || !adaGoal) {
        throw new Exception("Input harus memiliki Z dan O.");
    }
}

boolean isGoal(State s) {
    return s.row == goalRow && s.col == goalCol && s.nextNumber > maxNumber;
}

SlideResult slide(State state, Direction dir) {
    int r = state.row;
    int c = state.col;
    int nextNumber = state.nextNumber;
    int moveCost = 0;

    while (true) {
        int nr = r + dir.dx;
        int nc = c + dir.dy;

        if (nr < 0 || nr >= n || nc < 0 || nc >= m) {
            return null;
        }

        if (grid[nr][nc] == 'X') {
            break;
        }

        if (grid[nr][nc] == 'L') {
            return null;
        }

        r = nr;
        c = nc;
        moveCost += cost[r][c];

        char cell = grid[r][c];

        if (Character.isDigit(cell)) {
            int angka = cell - '0';

            if (angka == nextNumber) {
                nextNumber++;
```

```
                } else if (angka > nextNumber) {
                    return null;
                }
            }
        }

        if (r == state.row && c == state.col) {
            return null;
        }

        return new SlideResult(new State(r, c, nextNumber), moveCost);
    }

    void printBoard(State state) {
        char[][] temp = new char[n][m];

        for (int i = 0; i < n; i++) {
            temp[i] = grid[i].clone();
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (temp[i][j] == 'Z') {
                    temp[i][j] = '*';
                }
            }
        }

        temp[state.row][state.col] = 'Z';

        for (int i = 0; i < n; i++) {
            System.out.println(new String(temp[i]));
        }
    }
}

class SlideResult {
    State state;
    int moveCost;

    SlideResult(State state, int moveCost) {
        this.state = state;
        this.moveCost = moveCost;
    }
}
```

2.9.2 Direction.java

```
public enum Direction {
    U(-1, 0, "U"),
    D(1, 0, "D"),
    L(0, -1, "L"),
    R(0, 1, "R");

    int dx, dy;
    String symbol;
```

```
        Direction(int dx, int dy, String symbol) {
            this.dx = dx;
            this.dy = dy;
            this.symbol = symbol;
        }
    }
```

2.9.3. Main.java

```
    import java.util.*;
    import java.io.*;

    public class Main {
        public static void main(String[] args) {
            Scanner input = new Scanner(System.in);

            System.out.println("Ice Sliding Puzzle Solver");
            System.out.print("file input: ");
            String filename = input.nextLine().trim();

            try {
                Board board = new Board(filename);
                Solver solver = new Solver(board);

                System.out.print("pilih algoritma (UCS/GBFS/A*/BFS): ");
                String algorithm = input.nextLine().toUpperCase();

                if (algorithm.equals("astar")) {
                    algorithm = "A*";
                }

                String heuristic = "H1";

                if (algorithm.equals("GBFS") || algorithm.equals("A*")) {
                    System.out.print("pilih heuristic (H1/H2/H3): ");
                    heuristic = input.nextLine().toUpperCase();
                }

                SearchResult result = solver.solve(algorithm, heuristic);

                if (result.totalCost == -1) {
                    System.out.println("\nsolusi tidak ditemukan.");
                    System.out.println("waktu eksekusi: " + result.time + " ms");
                    System.out.println("iterasi: " + result.iterations);
                    input.close();
                    return;
                }

                System.out.println("\nsolusi yang ditemukan: " + String.join("", result.moves));
                System.out.println("cost solusi: " + result.totalCost);
                System.out.println("waktu eksekusi: " + result.time + " ms");
                System.out.println("iterasi: " + result.iterations);

                System.out.println("\ninitial");
                board.printBoard(result.states.get(0));
```

```
            for (int i = 1; i < result.states.size(); i++) {
                System.out.println("\nstep " + i + ": " + result.moves.get(i - 1));
                board.printBoard(result.states.get(i));
            }

            System.out.print("\nplayback? (ya/tidak): ");
            String playback = input.nextLine();

            if (playback.equalsIgnoreCase("ya")) {
                while (true) {
                    System.out.print("step, atau -1 untuk keluar: ");
                    int step = input.nextInt();

                    if (step == -1) break;

                    if (step >= 0 && step < result.states.size()) {
                        if (step == 0) {
                            System.out.println("\ninitial");
                        } else {
                            System.out.println("\nstep " + step + ": " + result.moves.get(step - 1));
                        }
                        board.printBoard(result.states.get(step));
                    } else {
                        System.out.println("step tidak tersedia.");
                    }
                }

                input.nextLine();
            }

            System.out.print("\nsimpan solusi? (ya/tidak): ");
            String save = input.nextLine();

            if (save.equalsIgnoreCase("ya")) {
                saveSolution("solusi.txt", board, result, algorithm, heuristic);
                System.out.println("solusi disimpan ke solusi.txt");
            }

        } catch (Exception e) {
            System.out.println("terjadi error: " + e.getMessage());
        }

        input.close();
    }

    static void saveSolution(String filename, Board board, SearchResult result, String
algorithm, String heuristic) throws Exception {
        PrintWriter out = new PrintWriter(new FileWriter(filename));

        out.println("algoritma: " + algorithm);
        if (algorithm.equals("GBFS") || algorithm.equals("A*")) {
            out.println("heuristic: " + heuristic);
        }

        out.println("solusi yang ditemukan: " + String.join("", result.moves));
```

```
            out.println("cost solusi: " + result.totalCost);
            out.println("waktu eksekusi: " + result.time + " ms");
            out.println("iterasi: " + result.iterations);
            out.println();

            for (int i = 0; i < result.states.size(); i++) {
                if (i == 0) {
                    out.println("initial");
                } else {
                    out.println("step " + i + ": " + result.moves.get(i - 1));
                }

                char[][] temp = new char[board.n][board.m];

                for (int r = 0; r < board.n; r++) {
                    temp[r] = board.grid[r].clone();
                }

                for (int r = 0; r < board.n; r++) {
                    for (int c = 0; c < board.m; c++) {
                        if (temp[r][c] == 'Z') {
                            temp[r][c] = '*';
                        }
                    }
                }

                State s = result.states.get(i);
                temp[s.row][s.col] = 'z';

                for (int r = 0; r < board.n; r++) {
                    out.println(new String(temp[r]));
                }

                out.println();
            }

            out.close();
        }
    }
```

2.9.4. Node.java

```
    public class Node {
        State state;
        Node parent;
        String move;
        int cost;
        double priority;

        public Node(State state, Node parent, String move, int cost, double priority) {
            this.state = state;
            this.parent = parent;
            this.move = move;
            this.cost = cost;
            this.priority = priority;
        }
    }
```

2.9.5. SearchResult.java

```
import java.util.ArrayList;

public class SearchResult {
    ArrayList<String> moves;
    ArrayList<State> states;
    int totalCost;
    int iterations;
    long time;

    public SearchResult(ArrayList<String> moves, ArrayList<State> states, int totalCost, int
iterations, long time) {
        this.moves = moves;
        this.states = states;
        this.totalCost = totalCost;
        this.iterations = iterations;
        this.time = time;
    }
}
```

2.9.6. Solver.java

```
import java.util.*;

public class Solver {
    Board board;

    public Solver(Board board) {
        this.board = board;
    }

    SearchResult solve(String algorithm, String heuristicName) {
        if (algorithm.equals("BFS")) {
            return bfs();
        } else {
            return informedSearch(algorithm, heuristicName);
        }
    }

    SearchResult bfs() {
        long startTime = System.currentTimeMillis();

        Queue<Node> queue = new LinkedList<>();
        HashSet<State> visited = new HashSet<>();

        State start = new State(board.startRow, board.startCol, 0);
        Node startNode = new Node(start, null, "", 0, 0);

        queue.add(startNode);
        visited.add(start);

        int iterations = 0;

        while (!queue.isEmpty()) {
            Node current = queue.poll();
            iterations++;
```

```
            if (board.isGoal(current.state)) {
                long endTime = System.currentTimeMillis();
                return makeResult(current, iterations, endTime - startTime);
            }

            for (Direction dir : Direction.values()) {
                SlideResult sr = board.slide(current.state, dir);

                if (sr != null && !visited.contains(sr.state)) {
                    visited.add(sr.state);
                    Node next = new Node(sr.state, current, dir.symbol, current.cost + sr.moveCost,
0);
                    queue.add(next);
                }
            }
        }

        long endTime = System.currentTimeMillis();
        return gagal(iterations, endTime - startTime);
    }

    SearchResult informedSearch(String algorithm, String heuristicName) {
        long startTime = System.currentTimeMillis();

        PriorityQueue<Node> pq = new PriorityQueue<>(new Comparator<Node>() {
            public int compare(Node a, Node b) {
                return Double.compare(a.priority, b.priority);
            }
        });

        HashMap<State, Integer> bestCost = new HashMap<>();

        State start = new State(board.startRow, board.startCol, 0);
        Node startNode = new Node(start, null, "", 0, 0);

        pq.add(startNode);
        bestCost.put(start, 0);

        int iterations = 0;

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            iterations++;

            if (board.isGoal(current.state)) {
                long endTime = System.currentTimeMillis();
                return makeResult(current, iterations, endTime - startTime);
            }

            for (Direction dir : Direction.values()) {
                SlideResult sr = board.slide(current.state, dir);

                if (sr == null) continue;

                int newCost = current.cost + sr.moveCost;
```

```
            if (!bestCost.containsKey(sr.state) || newCost < bestCost.get(sr.state)) {
                bestCost.put(sr.state, newCost);

                double h = heuristic(sr.state, heuristicName);
                double priority;

                if (algorithm.equals("UCS")) {
                    priority = newCost;
                } else if (algorithm.equals("GBFS")) {
                    priority = h;
                } else {
                    priority = newCost + h;
                }

                Node next = new Node(sr.state, current, dir.symbol, newCost, priority);
                pq.add(next);
            }
        }
    }

    long endTime = System.currentTimeMillis();
    return gagal(iterations, endTime - startTime);
}

double heuristic(State s, String heuristicName) {
    int dr = Math.abs(s.row - board.goalRow);
    int dc = Math.abs(s.col - board.goalCol);

    if (heuristicName.equals("H2")) {
        return Math.sqrt(dr * dr + dc * dc);
    } else if (heuristicName.equals("H3")) {
        return Math.max(dr, dc);
    } else {
        return dr + dc;
    }
}

SearchResult makeResult(Node goal, int iterations, long time) {
    ArrayList<String> moves = new ArrayList<>();
    ArrayList<State> states = new ArrayList<>();

    Node temp = goal;

    while (temp != null) {
        states.add(temp.state);

        if (temp.parent != null) {
            moves.add(temp.move);
        }

        temp = temp.parent;
    }

    Collections.reverse(states);
```

```
        Collections.reverse(moves);

        return new SearchResult(moves, states, goal.cost, iterations, time);
    }

    SearchResult gagal(int iterations, long time) {
        return new SearchResult(new ArrayList<>(), new ArrayList<>(), -1, iterations, time);
    }
}
```

2.9.7. State.java

```
import java.util.Objects;

public class State {
    int row, col;
    int nextNumber;

    public State(int row, int col, int nextNumber) {
        this.row = row;
        this.col = col;
        this.nextNumber = nextNumber;
    }

    public boolean equals(Object obj) {
        if (!(obj instanceof State)) return false;
        State other = (State) obj;
        return row == other.row && col == other.col && nextNumber == other.nextNumber;
    }

    public int hashCode() {
        return Objects.hash(row, col, nextNumber);
    }
}
```

2.9.8. Implementasi

Nama	Jenis	Deskripsi
Main.main(args)	Prosedur	Menjalankan program utama, menerima input nama file, memilih algoritma, memilih heuristic jika diperlukan, menampilkan hasil solusi, playback, dan menyimpan solusi ke file.
Board.readFile(filename)	Prosedur	Membaca file .txt, mengambil ukuran papan, isi map, matrix cost, posisi awal Z, posisi tujuan O, dan angka terbesar yang harus dilewati.
Board.slide(state, dir)	Fungsi	Menjalankan gerakan sliding ke arah tertentu sampai aktor berhenti sebelum batu X. Fungsi ini juga menghitung cost gerakan dan mengecek lava serta urutan angka.
Board.isGoal(state)	Fungsi	Mengecek apakah aktor sudah berada di titik tujuan O dan semua angka yang diperlukan sudah dilewati secara berurutan.

Board.printBoard(state)	Prosedur	Menampilkan kondisi papan sesuai posisi aktor pada state tertentu.
Solver.solve(algorithm, heuristicName)	Fungsi	Memilih algoritma pencarian yang akan dijalankan, yaitu UCS, GBFS, A*, atau BFS.
Solver.bfs()	Fungsi	Mencari solusi menggunakan BFS dengan queue. Algoritma ini memproses state berdasarkan urutan masuk dan mencari solusi dengan jumlah gerakan lebih sedikit.
Solver.informedSearch(algorithm, heuristicName)	Fungsi	Menjalankan UCS, GBFS, atau A* menggunakan priority queue. Perbedaan tiap algoritma ditentukan dari nilai prioritas node.
Solver.heuristic(state, heuristicName)	Fungsi	Menghitung nilai heuristic berdasarkan posisi aktor menuju tujuan. Heuristic yang digunakan adalah H1, H2, atau H3.
Solver.makeResult(goal, iterations, time)	Fungsi	Membentuk hasil akhir pencarian berupa urutan gerakan, daftar state, total cost, jumlah iterasi, dan waktu eksekusi.
Solver.gagal(iterations, time)	Fungsi	Mengembalikan hasil ketika solusi tidak ditemukan.
Main.saveSolution(filename, board, result, algorithm, heuristic)	Prosedur	Menyimpan hasil solusi ke dalam file .txt, termasuk algoritma, heuristic, cost, waktu eksekusi, jumlah iterasi, dan visualisasi langkah.
State.equals(obj)	Fungsi	Membandingkan dua state berdasarkan posisi baris, kolom, dan angka berikutnya yang harus dilewati.
State.hashCode()	Fungsi	Membuat kode hash agar objek State dapat digunakan di HashSet dan HashMap.

BAB 3 SCREENSHOT INPUT OUTPUT

3.1. TestCase 1

input1.txt
Input:

input1.txt

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

Output:

```
algoritma: A*
heuristic: H1
solusi yang ditemukan: DRU
cost solusi: 37
waktu eksekusi: 7 ms
iterasi: 7

initial
XXXXX
Xz*OX
X***X
X0*1X
XXXXX

step 1: D
XXXXX
X**OX
X***X
Xz*1X
XXXXX
```

```
step 2: R
XXXXX
X**OX
X***X
X0*zX
XXXXX

step 3: U
XXXXX
X**zX
X***X
X0*1X
XXXXX
```

3.2. TestCase 2

input2.txt
input:

```
input2.txt
5 5
XXXXX
XZ*OX
X***X
X0*1X
XXXXX
999 999 999 999 999
999 1 2 3 999
999 4 5 6 999
999 7 8 9 999
999 999 999 999 999
```

Output:

```
algoritma: A*
heuristic: H1
solusi yang ditemukan: RULUDRUR
cost solusi: 87
waktu eksekusi: 7 ms
iterasi: 13

initial
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
Xz**X*X
XXXXXXX

step 1: R
XXXXXXX
X0****X
X**X**X
```

```
X****OX
X1***LX
X**zX*X
XXXXXXX

step 2: U
XXXXXXX
X0****X
X**X**X
X**z*OX
X1***LX
X***X*X
XXXXXXX

step 3: L
XXXXXXX
X0****X
X**X**X
Xz***OX
X1***LX
```

```
X***X*X
XXXXXXX

step 4: U
XXXXXXX
Xz****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

step 5: D
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
Xz**X*X
XXXXXXX
```

```
step 6: R
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**zX*X
XXXXXXX

step 7: U
XXXXXXX
X0****X
X**X**X
X**z*OX
X1***LX
X***X*X
XXXXXXX
```

```
step 8: R
XXXXXXX
X0****X
X**X**X
X****zX
X1***LX
X***X*X
XXXXXXX
```

3.3. TestCase 3

input3.txt
input:

```
input3.txt
8 8
XXXXXXXX
X0*****X
X***X**X
X***L**X
X1****OX
X**X***X
XZ*****X
XXXXXXXX
999 999 999 999 999 999 999 999
999 1 2 3 4 5 6 999
999 7 8 9 999 1 2 999
999 3 4 5 999 6 7 999
999 8 9 1 2 3 4 999
999 5 6 999 7 8 9 999
999 1 2 3 4 5 6 999
999 999 999 999 999 999 999 999
```

Output:

```
PS C:\Users\LENOVO\Documents\Mirza\Stretegi Algoritma\Tucil3> java -cp bin Main
Ice Sliding Puzzle Solver
file input: test/input3.txt
pilih algoritma (UCS/GBFS/A*/BFS): A*
pilih heuristic (H1/H2/H3): H1

solusi tidak ditemukan.
waktu eksekusi: 6 ms
iterasi: 12
```

3.4. TestCase 4

input4.txt

Input:

```
6 6
XXXXXX
XZ***X
X*X**X
X0L1OX
X****X
XXXXXX
999 999 999 999 999 999
999 1 2 3 4 999
999 5 999 6 7 999
999 8 999 9 1 999
999 2 3 4 5 999
999 999 999 999 999 999
```

Output:

```
PS C:\Users\LENOVO\Documents\Mirza\Stretegi Algoritma\Tucil3> java -cp bin Main
Ice Sliding Puzzle Solver
file input: test/input4.txt
pilih algoritma (UCS/GBFS/A*/BFS): A*
pilih heuristic (H1/H2/H3): H1

solusi tidak ditemukan.
waktu eksekusi: 6 ms
iterasi: 9
```

3.5. TestCase 5

input5.txt
input:

```
input5.txt
9 9
XXXXXXXXX
X0******X
X***X***X
X***L***X
X1***2*OX
X***X***X
X***L***X
XZ******X
XXXXXXXXX
999 999 999 999 999 999 999 999 999
999 1 2 3 4 5 6 7 999
999 8 9 1 999 2 3 4 999
999 5 6 7 999 8 9 1 999
999 2 3 4 5 6 7 8 999
999 9 1 2 999 3 4 5 999
999 6 7 8 999 9 1 2 999
999 3 4 5 6 7 8 9 999
999 999 999 999 999 999 999 999 999
```

output:

```
PS C:\Users\LENOVO\Documents\Mirza\Stretegi Algoritma\Tucil3> java -cp bin Main
Ice Sliding Puzzle Solver
file input: test/input5.txt
pilih algoritma (UCS/GBFS/A*/BFS): A*
pilih heuristic (H1/H2/H3): H1

solusi tidak ditemukan.
waktu eksekusi: 4 ms
iterasi: 11
```

3.6. Analisis Hasil Percobaan

Dari hasil percobaan, setiap algoritma menunjukkan karakteristik yang berbeda. UCS cenderung menghasilkan jalur dengan total cost yang baik karena selalu memprioritaskan state dengan biaya terkecil. Namun, UCS dapat memerlukan iterasi lebih banyak karena proses pencariannya tidak diarahkan oleh heuristic.

GBFS dapat mencapai tujuan lebih cepat pada beberapa kasus karena memilih state yang terlihat dekat dengan tujuan. Akan tetapi, karena tidak mempertimbangkan total cost yang sudah ditempuh, jalur yang ditemukan bisa memiliki cost lebih besar dibandingkan UCS atau A*.

A* memberikan hasil yang lebih seimbang karena mempertimbangkan total cost dan heuristic secara bersamaan. Dengan heuristic yang sesuai, A* dapat mencari jalur secara lebih terarah tanpa mengabaikan biaya perjalanan.

BFS dapat menemukan solusi berdasarkan jumlah gerakan paling sedikit. Namun, karena tidak memperhatikan cost tile, solusi BFS belum tentu memiliki total cost yang paling rendah. Oleh sebab itu, BFS lebih tepat digunakan sebagai pembanding terhadap algoritma lain.

BAB 4 LAMPIRAN

4.1. Link Repository Program

Tautan Github:
https://github.com/Mirza114/Tucil3_13524114.git

4.2. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Mirza Tsabita Wafa'Ana